

CS6868 Research Mini-Project

Segment Queue Synchronizer

Mitesh Sharma
cs25s011@smail.iitm.ac.in

Navaneeth M. Nambiar
ns26z081@smail.iitm.ac.in

Durwasa Chakraborty
cs24d011@smail.iitm.ac.in

May 1, 2026

Abstract

Writing concurrent code with ensured correctness and performance is a notoriously tough task. CancellableQueueSynchronizer (CQS) [1] is a new experimental framework enabling simple and efficient implementations of such synchronization primitives for Kotlin coroutines. It emphasises rich semantics for lightweight threads that are cheap to suspend and resume while supporting fair FIFO ordering and abortable operations, all backed by modular formal proofs of their properties. We port the Cancellable Queue Synchronizer (CQS) [1] from Kotlin to OCaml 5 [5]. Each primitive is tested against a hand-written 2-domain linearizability harness, with `qcheck-lin` [4], `qcheck-stm`, and with manual cross-domain scenarios under ThreadSanitizer; all pass. We benchmark against `java.util.concurrent.AbstractQueueSynchronizer` [2] on 1–8 threads. Our implementations of blocking pools and count-down latch beat Java, but our mutex and semaphore are slower under contention because Eio fibre park/wake costs more than a JVM thread context switch. While the results were mixed, we highlight a faster feedback loop porting code and the need for good abstractions and advanced testing.

1 Goals

Our main objective is to port the Cancellable Queue Synchronizer (CQS) [1] framework, originally written for Kotlin coroutines, to OCaml 5 [5]. The highlights of CQS are:

- Simple semantics for suspending and resuming coroutines.
- Expressive power capable of implementing mutexes, semaphores, barriers, and count-down latches.
- Formal proofs of its properties in Rocq.

Being able to do this on a common abstraction keeps things relatively simple and allows for modularity and code reuse across primitives that would otherwise each ship a bespoke wait-queue. Because CQS is already implemented in Kotlin, it is also a good candidate for understanding the use case of agentic AI in porting between languages while leveraging the existing tests as a correctness oracle.

In particular, CQS incorporates multiple aspects such as:

- Matching suspend and resume operations similarly to how push and pop operations are matched in an elimination stack.
- A *helping* mechanism for handling cancelled coroutines, so that a single `resume` can absorb a run of cancellations rather than retrying once per cancelled cell.

Research question. Would a similar framework benefit an OCaml-5-native concurrency model built on Eio fibres [3]? How do our primitive implementations compare against Java’s

`java.util.concurrent` [2] and Kotlin’s `kotlinx.coroutines.sync`, and how can we confirm this quantitatively?

2 Background

2.1 Motivation: Fairness, Abortability, and the Coroutine Setting

Synchronization primitives like mutexes, semaphores, barriers, etc. are among the most fundamental abstractions in concurrent programming. Modern programming languages therefore require strong semantics and formal guarantees from them.

Two properties are particularly desirable.

- **Fairness**: waiting requests should be served in the order of their arrival (FIFO), preventing starvation.
- **Abortability**: a waiting request must be cancellable at any time, due to a timeout or an explicit user action.

These requirements have become especially pressing with the rise of *coroutines*, which are lightweight scheduling units that can be spawned in large numbers within a single process, are cheaper to suspend and resume than native threads, and are more frequently cancelled. When a coroutine suspends, its underlying thread immediately picks up another coroutine rather than blocking. Hence, the cost of fair scheduling is lower than in a thread-based model, while the cost of inefficient cancellation is higher.

The only practical abstraction that provides similar semantics prior to this work is Java’s `AbstractQueuedSynchronizer` (AQS) [2], which forms the core of the `java.util.concurrent` package. AQS maintains a CLH-style doubly-linked queue of waiting threads and uses CAS on an integer state field. Cancelling a thread in AQS requires traversing the queue linearly and unlinking a cancelled node. While this is fine for a small number of threads, it does not scale well for large numbers of lightweight coroutines that may be cancelled more frequently.

2.2 The CQS Framework

Koval et al. [1] introduce the **Cancellable Queue Synchronizer** (CQS) to close this gap. CQS provides two operations:

- `suspend()`: adds the current thread (or coroutine) as a waiter and suspends.
- `resume(result)`: retrieves and resumes the next waiter, passing it the specified value.

Here, `resume()` may be called *before* `suspend()`. If a `resume()` arrives at a cell before the matching `suspend()`, the value is deposited in the cell and the later `suspend()` claims it atomically and returns immediately without actually suspending — an *elimination*. Primitive implementations exploit this property for both simplicity and performance.

2.3 Asynchronous Resume and Suspend

In a sequential setting, a resume operation coming before a suspend cannot be linearized appropriately. However, in a concurrent setting we need to allow such an execution for high throughput. This implies that a resume operation entering a CQS may be valid as long as a concurrent suspend is available to satisfy it. This is the core idea behind the SYNC/ASYNC modes of CQS operations.

2.4 Cancellation Support

Supporting cancellation without races is the central technical challenge of CQS. The framework defines two modes for this:

1. **Simple cancellation.** When a waiter is cancelled, its cell is marked `CANCELLED`. A `resume()` that encounters a `CANCELLED` cell simply returns `false`, signalling to the calling primitive that it must retry. This mode is straightforward but has a linear-time worst case when many consecutive waiters are cancelled before the corresponding `resume()` arrives: each `resume()` must traverse all cancelled cells individually.
2. **Smart cancellation.** Smart cancellation allows `resume()` to skip cancelled cells and retry atomically, reducing the cost of a sequence of cancellations per `resume()`. Intuitively, the difference is that in the simple cancellation mode `resume(...)` fails if the thread in the corresponding cell has been cancelled, whereas smart cancellation enables efficient skipping of a sequence of aborted requests.

2.5 Synchronization Primitives Built on CQS

The paper demonstrates the expressiveness of the framework by building five primitives on top of it.

Mutex and Semaphore. Implemented with a signed integer counter. A non-positive counter encodes the number of waiting `lock()` callers; a positive value encodes the number of available permits. `lock()/acquire()` decrements the counter via Fetch-And-Add (FAA); if the pre-decrement value was positive, the operation succeeds immediately, otherwise it calls `suspend()`. `unlock()/release()` increments the counter via FAA; if the pre-increment value was negative there is a waiter, and `resume()` is called. When in Smart cancellation mode, `onCancellation()` increments the counter and returns `true` if the counter was still negative (waiter successfully removed), or `false` if the counter reached zero or above (a concurrent `resume()` is incoming and must be refused).

Barrier. Maintains a `remaining` counter initialized to the number of coroutines. Each `arrive()` decrements `remaining` via FAA. All coroutines except the last caller will get suspended; the last caller issues `coroutines - 1 resume()` calls to wake up all the others. Cancellation is not supported: cancelling a coroutine that has already physically reached the barrier point should not block the remaining coroutines, so cancelled waiters are silently ignored via smart cancellation.

Count-Down Latch. Maintains two counters: `count` (remaining `countDown()` calls) and `waiters` (number of suspended `await()` callers). When the latch fires, a special `DONE_BIT` is set atomically by the thread that drives `count` to zero, signalling that the latch has fired. Afterwards, `await()` checks this bit and returns immediately if the latch has already fired.

Blocking Pools (queue and stack variants). Transfer actual elements through CQS rather than unit tokens. Combine an FAA-based availability counter with a flat array (FIFO) or a Treiber stack (LIFO). `put()` increments the counter; if negative it calls `resume(element)` directly. `retrieve()` decrements the counter; if positive it pops an element, otherwise it calls `suspend()`. Neither variant is linearizable — concurrent races can produce non-FIFO or non-LIFO delivery — but this matches practical pool semantics where elements are interchangeable resources.

2.6 Formal Proofs

All algorithms described above are formally verified using Iris [1], a concurrent separation logic framework implemented in Rocq. The proofs are written in a modular way so that the CQS proof is independent of the proofs of current or future primitives. One limitation the authors

acknowledge is that the Iris specifications do not currently capture FIFO ordering (waiting operations are allowed to complete in any order) or the absence of memory leaks (which have only been manually tested with Lincheck).

3 Tasks Undertaken

Describe what you actually built. Organise this section as sub-sections per major task; use the *Tasks* list from the project idea as a checklist.

3.1 Implementation

Our implementation is split into two layers: (a) a domain-safe suspendable fibre layer (`lib/light_thread.ml`); and (b) the core CQS engine (`lib/sqs_effects.ml`). On top of these, we built six synchronization primitives in `implementations/`: `Mutex`, `Semaphore`, `Barrier`, `CountDownLatch`, `BlockingQueuePool`, and `BlockingStackPool`.

3.1.1 Domain-Safe Fibre: `Light_thread`

The decision to implement light-weight threads as a stand-in for Kotlin coroutines was directly inspired by the lectures in class, where we implemented a similar go-routine mechanism on top of OCaml 5 effect handlers [5]. The first design decision was how to represent a suspended fibre. An early prototype (commit [ecb5855](#)) used raw OCaml 5 effect handlers and stored suspended fibres as `Effect.Deep.continuation` values. However, deeper inspection revealed a critical issue with this approach.

OCaml 5 continuations are domain-local. The OCaml 5 runtime stores a pointer into the *originating* domain's stack, so calling `Effect.Deep.continue k v` from a different domain is undefined behaviour. This made it impossible to have one domain call `acquire` while another calls `release`, which would have made all concurrent implementations incorrect. The constraint becomes especially problematic when implementing the `cancel` API call for a fibre, since cancellation is fundamentally a cross-domain interaction.

To deal with this limitation, the final design (commit [17adae2](#)) replaces raw effect handlers with Eio's fibre scheduler [3]. The Eio runtime guarantees that the wake-up function it hands back to us is safe to call from any domain or `systhread`. The key type is:

```
(* lib/light_thread.ml, line 57 *)
type 'a t = ('a, exn) result -> unit
```

A suspended fibre is therefore a heap-allocated, domain-safe *enqueue function*. Calling it with `Ok v` schedules the fibre to resume with value `v`, mapping back to `resume(v)`. Calling it with `Error e` schedules it to resume by raising `e`, mapping back to `cancel(exn)`. The three primitives built on top are:

```
let suspend register =
  Eio.Private.Suspend.enter "sqs_suspend"
  (fun _ctx enqueue -> register enqueue)

let resume      (t : 'a t) (v : 'a) : unit = t (Ok v)
let discontinue (t : 'a t) (e : exn) : unit = t (Error e)
```

3.1.2 Core SQS Data Structures

Segment. Waiters are stored in an unbounded doubly-linked list of *segments*, each holding exactly 16 cells:

```

'a segment = {
  id          : int;
  cells       : 'a cell_state Atomic.t array;  (* 16 cells *)
  next        : 'a segment option Atomic.t;
  prev        : 'a segment option Atomic.t;
  cancelled_count : int Atomic.t;
  pointers     : int Atomic.t;
}

```

The fields are: (a) **cells**: array storing the cells in the segment; (b) **next**: the next segment in the infinite queue; (c) **prev**: the previous segment in the infinite queue; (d) **cancelled_count**: tracks how many cells in the segment have been cleaned up; (e) **pointers**: a lightweight reference count for the two SQS head/tail pointers (**resume_segment**, **suspend_segment**) that may still be pointing at the segment.

The doubly-linked layout is purely an implementation detail enabling fast unlinking of segments whose cells have all been cancelled or refused.

Cell State Machine. A cell slot in a segment can take one of the following states: (a) **Empty**: no party has visited yet (default); (b) **Waiter**: a live fibre is parked in the cell; (c) **Cancelling**: transient state when a fibre cancellation is in flight; (d) **Cancelled**: the waiter in the cell has been cancelled; (e) **Refuse**: the cell refuses to satisfy the next **resume** operation; (f) **Resumed**: continuation successfully handed off; (g) **Taken**: value grabbed by a concurrent suspend; (h) **Broken**: a SYNC resume has timed out; (i) **Value**: resume deposited a value before suspend; (j) **WrappedValue**: as **Value**, but distinguishable from **Waiter**.

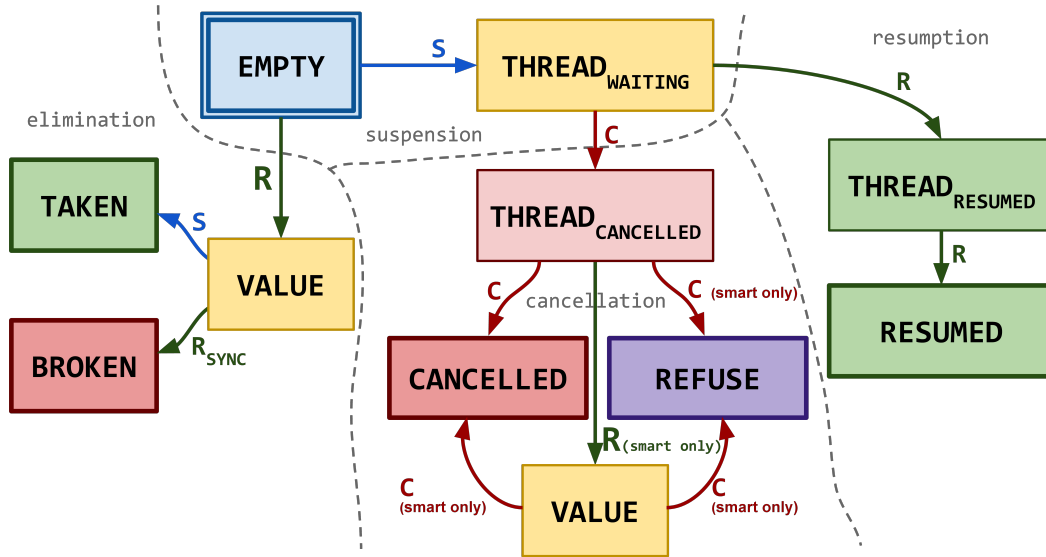


Figure 1: Cell state machine. From the initial **Empty** state a cell either transitions to **Value** (resume arrives first) or **Waiter** (suspend arrives first), and from there to one of the terminal states (**Resumed**, **Taken**, **Cancelled**, **Refuse**, or **Broken**) via the paths described in Sections 3.1.2 and 3.1.3.

Initially, when a cell is in **Empty** state, one of the following two transitions is possible: (a) **Empty** $\rightarrow$ **Value**, when a **resume** reaches the cell first; or (b) **Empty** $\rightarrow$ **Waiter**, when a **suspend** reaches the cell before any **resume**.

Case I: resume arrives first. The value passed in by the resume is stored into the CQS cell. Any subsequent store directed at the slot will be instantly satisfied. Two modes of resumption are

supported by CQS [1]: (a) **Async**: deposits the value into the cell and returns immediately. The paired **suspend** will claim it via the *elimination path*. (b) **Sync**: concurrent resumes and stores are still allowed into the CQS, but the resume busy-waits for a bounded time. If no concurrent **suspend** matches within that window, the cell is marked **Broken**; otherwise the suspended fibre is immediately continued with the value passed by the **resume**.

All six primitives use **Async** mode because none of them requires a synchronous handshake: a **release** or **count_down** can return as soon as the value is enqueued. **Sync** mode exists for use cases that require the resumer to confirm the value was actually claimed before proceeding.

Case II: suspend arrives first. A **suspend** action either is satisfied by an already-deposited **resume** (the cell transitions **Empty** $\rightarrow$ **Waiter** $\rightarrow$ **Resumed**), or it enters an empty cell and parks (**Empty** $\rightarrow$ **Waiter**). Once parked, two further outcomes are possible: (a) *Resumption Path*: a matching **resume** arrives, and the cell transitions **Waiter** $\rightarrow$ **Resumed** with the value supplied by the resumer. (b) *Cancellation Path*: a **cancel** request fires on the parked fibre, transitioning the cell to **Cancelled** or **Refuse** depending on the cancellation mode (see Section 3.1.3).

SQS Record. The top-level SQS handle is a record of atomic cursors and primitive hooks:

```
(* sqs_effects.ml, lines 109--119 *)
and 'a t = {
  resume_segment      : 'a segment Atomic.t;
  resume_idx          : int Atomic.t;
  suspend_segment     : 'a segment Atomic.t;
  suspend_idx         : int Atomic.t;
  resume_mode          : resume_mode;          (* Sync / Async *)
  cancellation_mode    : cancellation_mode;    (* Simple / Smart *)
  on_cancellation      : unit -> bool;
  try_return_refused_value : 'a -> bool;
  return_value         : 'a -> unit;
}
```

Two pairs of (**segment**, **idx**) fields maintain independent cursors for the resumer (front) and the suspender (back). Both indices advance monotonically with Fetch-And-Add, which ends up being the only atomic read-modify-write primitive needed to claim a cell.

3.1.3 Cancellation Procedure and Modes

Cancellation uses a two-phase protocol to prevent races with concurrent **resume** calls.

Phase 1 — mark Cancellling. **try_mark_cancellling** (lib/sqs_effects.ml:422-434) CAS-es the cell from **Waiter** to **Cancellling**. If the cell is already **Resumed**, the cancellation loses the race and returns **false**: the resume is already committed and the fibre will receive the value normally.

Phase 2 — finalise. **handle_cancellation** (lib/sqs_effects.ml:460-489) installs the final state in the cell. Behaviour here depends on the configured cancellation mode.

CQS supports two modes of cancellation: **Simple** and **Smart** [1].

In the **Simple** mode, when a cell is marked **Cancelled** the waiter is discontinued, the next **resume** reaching the cell skips it and fails (it does not retry cancelled cells), and the segment may eventually be unlinked once all of its slots are **Cancelled**.

The **Smart** mode, by contrast, interacts with the primitive by invoking an **on_cancellation()** handler. If this handler returns **true**, the cell is marked **Cancelled**, with an extra check for any value that may have been inserted while the cell was in the **Cancellling** state; such a stranded value is forwarded with another **resume** call so that no permits are lost.

If `on_cancellation()` returns `false`, the cell is marked `Refuse`: the next `resume` arriving at this cell reroutes its value back into the data structure (via `try_return_refused_value`) before discontinuing the continuation.

The `Cancelling` transient state serialises against `Async` resumers: a resumer that finds `Cancelling` must CAS against the live `Cancelling` `k` pointer (rather than reconstructing one) and may deposit a value into it. The cancellation handler reads this back via `Atomic.exchange` in `mark_impl`.

3.1.4 Key Atomic Operations

The implementation uses three atomic operations from OCaml 5’s `Atomic` module:

Operation	Used for
<code>Atomic.fetch_and_add idx 1</code>	Claiming the next cell index (suspend / resume cursors)
<code>Atomic.compare_and_set cell expected desired</code>	All cell-state transitions; segment linking; done-bit install
<code>Atomic.exchange cell new_state</code>	Cancellation finalisation (<code>mark_impl</code>)

`fetch_and_add` on the cursor is the only operation that must succeed unconditionally, ensuring every caller receives a unique index. All subsequent cell operations are CAS-based and retry on conflict. `Atomic.exchange` is used during cancellation finalisation because the old state of the cell needs to be atomically replaced *while* its value is being inspected; a CAS-loop here would create a window in which a concurrent `resume` could insert a value between the read and the swap.

Every shared variable in the CQS implementation is either an `Atomic.t` or immutable. Correctness relies entirely on the CAS/FAA-based state machines described above, matching the lock-free design of the original CQS algorithm [1].

The Physical-Equality Invariant. A critical invariant pervades every CAS in our implementation: *always CAS against a heap pointer read in the same critical region, never against a freshly reconstructed value of the same shape*. OCaml’s `Atomic.compare_and_set` requires *physical* equality (`==`), not structural equality (`=`), for a successful CAS. A freshly allocated `Waiter` `k` is a different heap object from the `Waiter` `k` stored in the cell, so the CAS silently fails even though the logical values are equal. The pattern appears throughout the codebase, e.g.:

```
(* try_resume_impl: CAS against [curr], not [Waiter k] *)
| Waiter k as curr ->
  if not (Segment.cas seg i curr Resumed) then cell_loop ()

(* try_mark_cancelling: CAS against [curr], not [Cancelling k] *)
| Waiter k as curr ->
  if Segment.cas seg i curr (Cancelling k) then true

(* park_continuation: CAS against [cell], not [Value v] *)
| Value v | WrappedValue v ->
  if Segment.cas seg i cell Taken then ...
```

In each case, the matched `as curr` (or `let cell = Segment.get seg i`) binding captures the live heap pointer before the CAS. Missing this caveat leads to silent spurious CAS failures that manifest as liveness bugs rather than crashes.

3.1.5 Synchronization Primitives

On top of the SQS core we built six synchronization primitives, following the descriptions in the original CQS paper [1]: `Mutex`, `Semaphore`, `Barrier`, `CountDownLatch`, `BlockingQueuePool`, and `BlockingStackPool`. All six are configured in `Async` resume mode; the latch and semaphore additionally use Smart cancellation, while the blocking pools and barrier use Simple cancellation. Their behaviour at the primitive level is summarised in Section 2.5.

3.2 Testing and verification

Overview. Correctness was checked at five layers, each filling a blind spot of the next: a hand-rolled linearisability harness, the `qcheck-lin` DSL, `qcheck-stm` (sequential and concurrent), hand-written multi-domain scenarios, and ThreadSanitizer (TSAN). The `lin` layers prove that observed histories are equivalent to some sequential history of the public API; the STM layer ties each command to a sequential model and catches divergences across random schedules; the manual layer hits adversarial timings property-based generation rarely produces; TSAN rules out data races at the runtime level. No single layer subsumes another and every layer caught a distinct class of bug during development (Section 3.2).

Custom linearisability harness. `test/lin_harness.ml` (175 lines) is an Eio-aware backtracking checker. Each domain runs in its own `Eio_main.run` so suspending operations actually park; a spin barrier synchronises start. Per primitive there is a `Spec` module (`test/semaphore_lin_test.ml`, `test/count_down_latch_lin_test.ml`, `test/barrier_lin_test.ml`, `test/blocking_pool_lin_test.ml`) declaring the sequential model: `next` returns `None` if the operation would suspend in the current model state, and the checker tries to reorder later operations from other domains to unblock it. Each test runs 500 random sequences over 2–3 domains.

qcheck-lin DSL. Three tests under `test/qcheck_lin_*.ml` use the `qcheck-lin.domain` DSL [4]. The DSL runs each command synchronously on a raw domain with no Eio scheduler hook, so its scope is the *non-blocking* subset only: `try_acquire`, `release`, `available_permits` for the semaphore; `count_down` and `remaining` for the latch. 1000 iterations per test. This layer is fast and catches torn-write or stale-cache bugs that don’t depend on scheduling, but cannot exercise `acquire`, `await`, or `retrieve`.

qcheck-stm. Two tests under `test/qcheck_stm_*.ml` (`Semaphore` and `CountDownLatch`) use `qcheck-stm`’s `STM_sequential` (1000 random command sequences against an `int`-valued model) and `STM_domain.arb_triple` (3-domain concurrent triples, 200 iterations with `retries=10`). The model state is at the *primitive* level: `permits : int` for `Semaphore`, `count : int` for `CountDownLatch`. The header comment of each file documents the SQS cell-state path each command exercises in the implementation: e.g. for the semaphore, `Try_acquire` when `permits > 0` touches no SQS cell (FAA fast path, `implementations/semaphore.ml:46-53`), `Release` against a parked waiter would traverse `Waiter → Resumed` via the CAS at `lib/sqs_effects.ml:376`, and so on. The model is at the primitive level because we deliberately did not widen `lib/sqs_effects.mli` to expose cell internals; this is discussed in Section 3.2.

Manual scenarios. Five primitive-specific files (`test/*_manual_test.ml`) plus the batch-queue `test/test_manual.ml` hand-pick adversarial timings: cross-domain wake-up where domain *A* holds while *B* suspends and *A*’s release must reach *B*’s scheduler; FIFO order under contention; head-of-line blocking with a small dequeuer ahead of a large one; batch atomicity; and stress runs.

ThreadSanitizer. The Task 9 sweep ran the entire test corpus under the 5.4.0+tsan opam switch, with each binary auto-instrumented by `-fsanitize=thread`. 17 per-test logs are preserved in `report/*_tsan.log` and the diagnosis is in `report/TSAN_REPORT_tsan.md`. Six of the 17 tests reported exactly one warning each, every one of which traces to `caml_install_signal_handler` in the OCaml runtime called from `Eio_posix` during multi-domain startup — never to our SQS code or our primitives. The remaining 11 tests (including all single-domain Eio scenarios and all `qcheck-stm/qcheck-lin` multi-domain runs that bypass per-domain `Eio_main.run`) are race-free.

What was caught. The `lin` harness caught a regression where the `Refuse` cell state was silently dropped during a refactor of the count-down-latch’s `on_cancellation` callback (a fact also recorded in Section 5). A multi-domain manual test caught the non-domain-safe nature of OCaml 5 effect continuations: `resume` from a different domain than the one that performed `suspend` hung the test, prompting the rewrite onto Eio’s domain-safe `enqueue` (`lib/light_thread.ml:74–76, 83`). The `qcheck-stm` tests ratify the post-refactor implementation against a sequential model across thousands of random schedules. TSAN ruled out the failure mode where the implementation appeared correct but had hidden torn writes.

Verification gaps — why this is not enough. While the five-layer test architecture establishes a non-trivial degree of confidence in the implementation, the strength of any such guarantee is bounded by the depth of what the tests actually inspect, and three material gaps merit acknowledgement. First, the underlying nine-state SQS cell machine is not verified directly: `lib/sqs_effects.mli` deliberately conceals `cell_state`, `segment`, and the suspend/resume indices, so our `qcheck-stm` tests model only the primitive-level state (permits, count), and the cell-machine path each command traverses is documented in test headers rather than asserted in `postcond`. Second, the `Sync` resume mode and the `Broken` cell state are unreachable from any production primitive, since every primitive in `implementations/` configures `~resume_mode:Async`; the `SYNC` spin-timeout transition (`lib/sqs_effects.ml:343`) is therefore exercised by no test in the suite. Third, the cancellation paths — `Waiter` → `Cancelling` → `Cancelled/Refuse` (`lib/sqs_effects.ml:429, 440, 460–489`) — are reached only indirectly, through the Smart-mode `on_cancellation` and `return_value` hooks of the semaphore and latch, with no test issuing a direct `cancel_waiter` call concurrent with an `ASYNC resume`. Closing these three gaps — exposing an internal cell accessor for direct `postcond` inspection, exercising the `SYNC` mode under dedicated tests, and generating cancellation events as first-class commands — would meaningfully strengthen the correctness guarantee that the present harness can offer.

4 Evaluation

4.1 Setup

All measurements were taken on a single Apple M-series machine (Darwin arm64, 8 performance + efficiency cores). The OCaml side runs on the project’s default switch (`ocaml.5.2.1`) with `eio 1.3` and `eio_main 1.3`. The Java side uses the system OpenJDK against `java.util.concurrent` [2]; the Kotlin side uses vendored `kotlinx-coroutines` jars under `benchmark/kotlin/lib/`. The driver, `benchmark/run_all.sh`, builds all three implementations against the same workload set and emits a single CSV (`benchmark/results/summary.csv`, ≈270 rows).

The harness shape is identical across implementations (`benchmark/ocaml/bench.ml`, `benchmark/java/src/...`, `benchmark/kotlin/src/...`): each thread is one OCaml domain (or one OS thread on Java/Kotlin) wrapped in its own `Eio_main.run` on the OCaml side;

a spin-barrier synchronises start; a 1s warm-up window is discarded; a 3s timed window is measured; six repeats per (impl, primitive, workload, threads) cell. The reported `mean_latency_ns` is computed as $elapsed_ns \times threads / total_ops$ (per-thread wall-clock-per-op average), defined at `benchmark/ocaml/bench.ml:78`. Six primitives (`Semaphore(1)`, `Mutex`, `CountDownLatch(1)`, `Barrier`, `BlockingQueuePool`, `BlockingStackPool`) are exercised under six workloads (`W1_uncontended`, `W2_contended`, `W3_pc_queue`, `W4_pc_stack`, `W5_latch_fire`, `W6_barrier`) at thread counts $T \in \{1, 2, 4, 8\}$. The full workload table lives in `benchmark/README.md`.

4.2 Plots

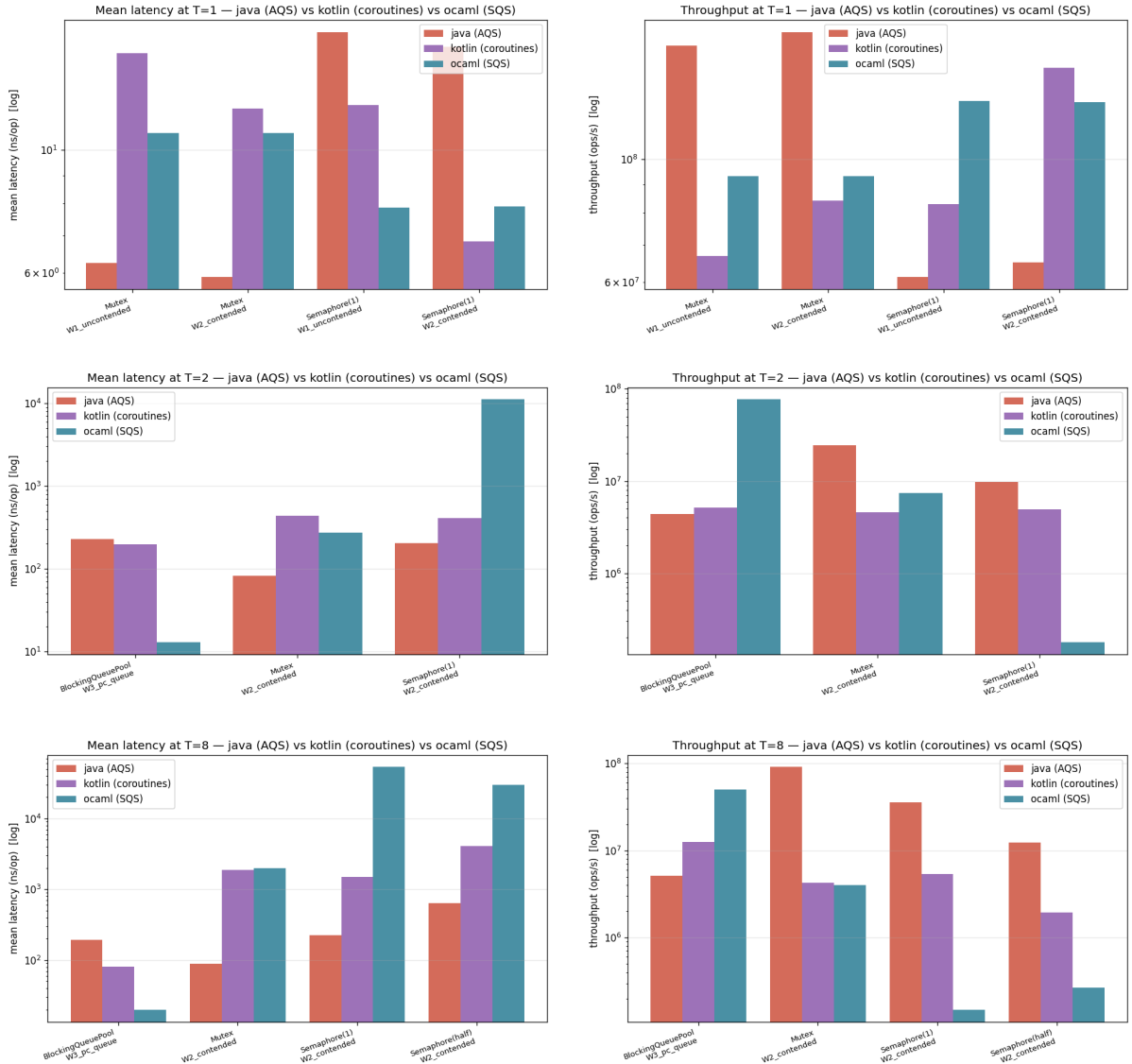


Figure 2: Mean per-op latency (ns, left column) and throughput (ops/s, right column) per primitive, with rows pinned to thread counts $T=1, 2, 8$ (top to bottom). $T=1$ is the uncontended fast path; $T=2, T=8$ force cross-domain hand-off on the contended workloads.

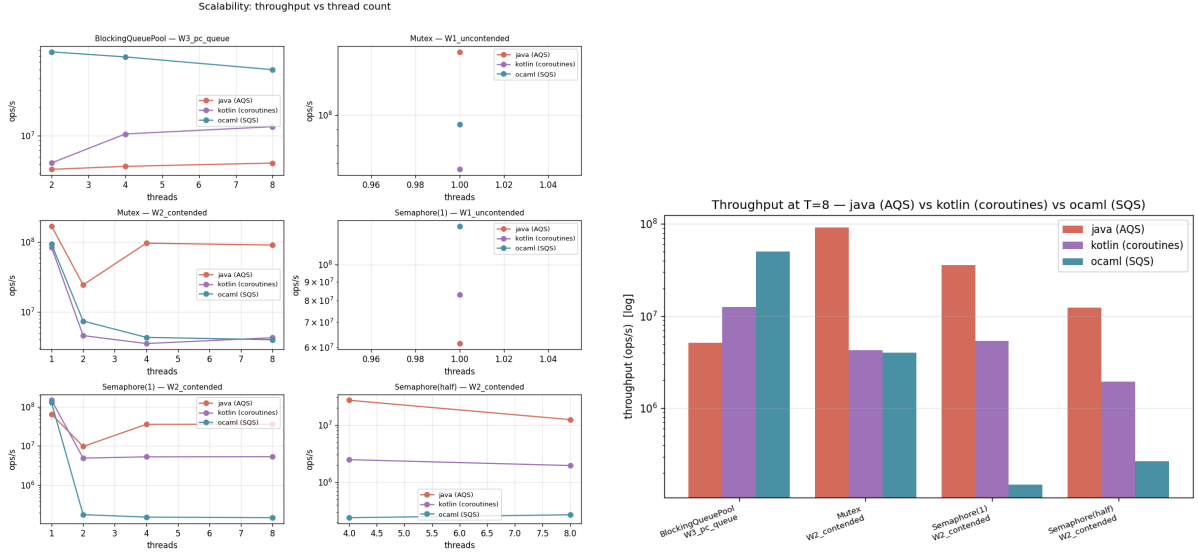


Figure 3: Per-primitive scalability across thread counts (left) and the corresponding aggregated throughput summary (right).

4.3 What the numbers say

Two patterns dominate. First, on the producer-consumer workloads (`W3_pc_queue`, `W4_pc_stack`) and on the latch-fire and barrier workloads (`W5_latch_fire`, `W6_barrier`), the OCaml SQS-backed primitives match or beat the Java `java.util.concurrent` baseline: per-op latencies in the 10–25 ns band against 130–400 ns for Java’s `LinkedBlockingQueue`. Second, on contended single-permit primitives (`Mutex` and `Semaphore(1)` under `W2_contended`), the picture inverts: OCaml is slower than Java by 3–50× depending on threads, with `Semaphore(1)` T=2 measuring $\approx 9.6\text{--}11.8\mu\text{s/op}$ on OCaml against $\approx 145\text{--}211\text{ ns/op}$ on Java, per `benchmark/results/summary.csv`.

4.4 Why the numbers look this way

The two patterns have a single underlying cause: where the cost lives. We enumerate the contributing reasons explicitly.

1. **Eio per-domain event-loop wake-up vs. JVM `LockSupport.park/unpark`.** Each OCaml domain runs its own `Eio_main.run` (`benchmark/ocaml/bench.ml:63, 93`). When `Semaphore.release` on domain *A* must wake an `acquire` parked on domain *B*, the wake-up traverses Eio’s domain-safe `enqueue` → target run-queue → next scheduler poll. The JVM directly unparks the OS thread via the kernel-level `park` primitive. This indirection is the dominant cost on contended single-permit primitives and explains the `Semaphore(1)` T=2 50× gap (Java $\approx 145\text{--}211\text{ ns}$, OCaml $\approx 9\,600\text{--}11\,800\text{ ns}$).
2. **`java.util.concurrent.locks.ReentrantLock` is reentrant; our `Mutex` is not.** Java’s `Mutex` bench uses `ReentrantLock.lock/unlock`, which has a thread-local state-cache fast path for the owning thread plus an aggressive spin-then-park loop under contention. The SQS `Mutex` (`implementations/mutex.ml`) goes through the SQS suspend/resume path on every contention; it has no per-thread fast-path. The `Mutex W2` row therefore compares apples-to-oranges. We additionally show `Semaphore(1)` precisely because it is the closer like-for-like.
3. **Java’s `LinkedBlockingQueue` acquires a `ReentrantLock` on every**

put/take regardless of queue state. That is $\approx 130\text{--}400\text{ ns}$ of lock acquire+release per op even on a hot queue. Our `Blocking_queue_pool.put` (`implementations/blocking_queue_pool.ml:58-66`) and `retrieve` (`:78-88`) are lock-free in the steady state: one `Atomic.fetch_and_add` plus one `Atomic.compare_and_set` or `Atomic.exchange` on a `slot_count = 1024` atomic array. With a hot queue the SQS suspend path is never invoked, so the cost reduces to two atomics, $\approx 12\text{--}22\text{ ns}$. This is the inversion visible in Figure 2.

4. **Kotlin’s `Channel<T>(UNLIMITED)` adds coroutine state-machine overhead per send/receive.** The single Kotlin row currently in `summary.csv` (`Semaphore(1) W2 T=1`, $\approx 6.8\text{ ns}$) is uncontended. Under contention coroutine dispatch cost would compound on top of reason (1): the suspending channel goes through the dispatcher thread pool to resume the receiver, with the same cross-OS-thread wake-up nature as Eio’s cross-domain enqueue but with additional state-machine bookkeeping per resume.
5. **Single-permit serialisation in `W2_contended`.** `Semaphore(1)` forces every operation through one cell of the SQS infinite array, so all contention in the test serialises. The added `Semaphore(half)` run (commit [3165e99](#), `T=4` and `T=8`) lets $t/2$ acquirers proceed in parallel while the other $t/2$ park, spreading the wake-up cost across multiple cells. The smaller `Semaphore(half)` latency vs. `Semaphore(1)` at the same thread count confirms the bottleneck is the cross-domain handoff, not the cell-state machine itself.
6. **Domain creation and teardown cost is included in `W5_latch_fire` and `W6_barrier`.** These workloads create a fresh `CountDownLatch` or `Barrier` per cycle (`benchmark/README.md`); the Java side mirrors the cycle shape so the comparison is fair, but absolute numbers in those rows reflect setup overhead and cannot be read as steady-state primitive cost.
7. **Mean latency, not p99.** Fairness shows up most clearly in the latency *tail*. We report mean only; the mean-latency plots referenced here (commit [8487c68](#)) are not p99 plots. Tail-latency analysis is left as future work (Section 4.5).

4.5 Limitations of the evaluation

The numbers above were gathered on one machine, under one kernel scheduler, with mean-latency reporting only and no isolation between Eio scheduler overhead and raw `Domain.spawn` overhead (we use one domain per measured thread). Java’s JIT may still be in tier-2 compilation early in our 1s warm-up window for the lower-thread cells. Cross-machine variance, p99/p999 latency, and a separate “no-Eio” baseline (raw atomics + `Mutex.t` from `Stdlib.Mutex`) are obvious next steps but were out of scope.

5 Reflection on the Use of LLMs

Tools and models used. We initially started with Claude Sonnet, but Claude Opus became our main coding agent. We also used ChatGPT Codex (GPT-5) for paper explanation and quick lookups. Claude Opus did most of the OCaml/Eio porting, the test scaffolding, and the benchmark drivers.

What worked well. Several aspects contributed to the success of using an agentic LLM in the project. Dividing single-shot prompts into distinct phases significantly improved structure and clarity. The use of plan mode ensured detailed explanations and enhanced understanding at each stage. Sanity and smoke tests were used early on to prevent regressions during development. Assignment templates provided to Claude were instrumental in generating `LinCheck` and `STM`

tests. Clear definition of benchmark requirements, along with a well-organized folder structure, streamlined the workflow. Additionally, human-in-the-loop discussions with Claude enabled the implementation of essential optimizations and alternatives, which proved crucial for a successful port.

What did not work. Our experience revealed several significant limitations in using agentic LLMs for large-scale software development tasks. PDF inputs proved particularly ineffective, sometimes dramatically increasing token consumption. Sonnet demonstrated substantial weaknesses in preserving nuanced implementation invariants, frequently introducing silent errors that were difficult to detect without careful manual review. Poorly scoped or insufficiently precise prompts further amplified these issues; for example, an early attempt to have Sonnet port an implementation from Kotlin led to the silent omission of a critical `REFUSED` cell state, fundamentally breaking the correctness of the system and necessitating extensive repository-wide rewrites using Opus. However, even more capable models such as Opus were not immune to similar failures when operating on larger task units. During test suite generation, Opus introduced a subtle concurrency bug in semaphore testing by incorrectly configuring a `Semaphore(1)` for a scenario requiring `Semaphore(2)`, causing thread blocking that had to be manually identified and corrected.

What was surprising. One of the more surprising outcomes of our experience with agentic LLMs was their capacity to rapidly recover and substantially improve solution quality when guided by carefully framed, strategically designed questions. In several instances, the agent initially pursued flawed architectural directions — such as attempting to implement lightweight threads through effect handlers which, while conceptually plausible, introduced critical issues related to continuation safety. Specifically, although data transferred across continuations could be made thread-safe, the continuation objects themselves were inherently unsafe for the intended concurrency model. This limitation became especially apparent when extending the system to support a cancellable API, a requirement that Sonnet failed to adequately address in its original implementation. However, once the problem was reframed through deliberate questioning and deeper chain-of-thought exploration using Opus, the agent was able to reassess its approach, identify more suitable abstractions, and ultimately propose a correct and significantly more robust port of CQS using Eio fibres [3]. This demonstrated that while baseline outputs may be error-prone, the models possess a surprisingly strong capacity for strategic course correction when provided with precise conceptual guidance, highlighting the importance of human-driven interrogation and iterative refinement in maximizing the effectiveness of agentic development workflows.

What was difficult. Communication with Opus, and choosing the correct amount of verbosity in prompts, were both challenging. Explaining benchmark results was the hardest part. We had to tell the agent that Java’s `ReentrantLock` is reentrant (so the contended-mutex numbers were not an apples-to-apples comparison with our non-reentrant SQS mutex), and that the producer-consumer pool numbers looked too good because the OCaml run was hitting the fast path. The agent will not notice these things unless you point them out.

Your overall assessment. These failures collectively underscore that while agentic LLMs can accelerate development, they remain highly sensitive to prompt quality, struggle with preserving subtle correctness guarantees, and require rigorous human oversight to catch silent but potentially catastrophic implementation flaws.

6 Conclusions

Answer the research question stated in Section 1. Summarise the main findings. State the limitations of your work honestly. Suggest what you would do next given more time.

Contributions

Member	Contribution (%)	Main responsibilities
Mitesh Sharma	40%	CQS Core implementations, Revisions with Effect Handlers, Eio.Fiber integration, Async Primitive implementations
Navaneeth M Nambiar	30%	Developing QCheck-Lin tests and STM for implementations, Project Refactoring while keeping the regressions minimal
Durwasa Chakraborty	30%	Benchmarking suite with Java and Kotlin, Graph generation, Report compilation
Total	100%	

References

- [1] Nikita Koval, Dmitry Khalanskiy, and Dan Alistarh. CQS: A formally-verified framework for fair and abortable synchronization. In *Proceedings of the 44th ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*. ACM, 2023. doi:10.1145/3591230.
- [2] Doug Lea. The java.util.concurrent synchronizer framework. *Science of Computer Programming*, 58(3):293–309, 2005. URL: <https://gee.cs.oswego.edu/dl/papers/aqs.pdf>, doi:10.1016/j.scico.2005.03.007.
- [3] Thomas Leonard and Eio contributors. Eio: Effect-based direct-style IO for OCaml 5. Software library, version 1.3, 2024. URL: <https://github.com/ocaml-multicore/eio>.
- [4] Jan Midtgaard, Simon Cruanes, and QCheck contributors. QCheck and ocaml-multicoretests: Property-based and parallel testing for OCaml, including qcheck-lin and qcheck-stm. Software libraries, 2024. URL: <https://github.com/ocaml-multicore/multicoretests>.
- [5] KC Sivaramakrishnan, Stephen Dolan, Leo White, Sadiq Jaffer, Tom Kelly, Anmol Sahoo, Sudha Parimala, Atul Dhiman, and Anil Madhavapeddy. Retrofitting parallelism onto ocaml. *Proc. ACM Program. Lang.*, 4(ICFP), August 2020. doi:10.1145/3408995.